

When Financial Agents Code: Auditing Spurious Alpha, Data Leakage, and Execution Errors with Guarded Domain Libraries

Anonymous ACL submission

Abstract

Financial agents increasingly write the quantitative code behind their conclusions, yet a plausible backtest can conceal future information, a misapplied corporate action, or a validation claim that never executed. We present FIN-SKILLS, a financial library of 129 skills, 48 algorithm-backend catalog entries, and 36 executable research guards that turn dated domain conventions into structured computation and artifact-bound audit records. Across three seeded worlds, the guard suite detects all 36 of 36 planted defects with no false alarms on the clean references, and 8 of 8 numerical checks meet their stated tolerances. In a frozen paired study of first-submission code generation with two open code models, none of 48 programs meets the pre-registered validity criteria: 37 responses break the code-only format and every executed program fails numerical and split checks. With the first code fence extracted post hoc, 37 programs execute but only 1 is valid; 24 programs mishandle cumulative split factors and 15 use same-session or future prices; documentation alone produced no library calls. In a frozen follow-up with three attempts, enforcing lag and split checks cut accepted-but-wrong programs from 22 of 24 to 2, and to 0 with an enforced library call, but did not raise valid programs (1, 1, and 0 of 24). The checks prevent silent errors from being delivered; they do not make small models write correct code. HiSTrim and persistent memory are optional integrations whose results are reported separately.

1 Introduction

A financial agent can produce a convincing research report while joining a disclosure to dates before publication, treating a stock split as a return, or reporting a check that never executed. These failures can create apparent alpha without a usable investment signal, and they increasingly arise in code the agent writes itself. Prevent-

ing them requires financial conventions to be expressed as executable input requirements and tests, alongside the algorithms that consume the data.

FIN-SKILLS provides this infrastructure. Its domain skills identify appropriate libraries and dated failure modes; its numerical interfaces expose parameter and data contracts; and its research guards check availability times, corporate actions, validation splits, costs, and other declared assumptions. Structured findings distinguish a detected defect from a missing input or a successful check. Artifact-bound execution records allow a controller to enforce those checks rather than trust an agent’s prose.

We ask whether language models can reliably build quantitative computations without such a library, and what the library’s checks and primitives establish. E1 measures whether the guard suite detects known defects without flagging clean references. E2 measures whether agents discover and execute the interfaces and whether submitted artifacts meet the audit contract. E3 compares numerical primitives with independent references. E4 freezes a paired code-generation study in which two open code models implement four allocation and signal rules with and without the library’s documentation, graded by an independent executable oracle, then repeats the tasks with three attempts while a controller enforces the library call and lag and split checks. First submissions are almost never valid, and the dominant errors (re-cumulated split factors and same-session prices) execute without raising an error. Enforcement stops most wrong programs from being accepted but does not produce more correct ones, which separates the library’s auditing value from any claim of better generation.

The distinction matters for interpreting an earlier portfolio experiment. That experiment exposed a price-compatible subset of algorithms on one short, anonymized currency path, with-

out mandatory guard enforcement. It is an exploratory component ablation of algorithm access, not a comparison of the complete guarded library against an unguarded system. We retain its returns and execution traces, but do not use its losses to infer either that financial tools are ineffective or that memory would repair those losses.

Our contributions are (1) a library of 129 financial skills, 48 algorithm-backend catalog entries, and 36 executable research guards; (2) structured computation and artifact-bound verification interfaces connecting financial assumptions to actual execution; (3) evidence on planted-defect detection, execution behavior, and numerical parity; and (4) frozen, hash-bound audits of financial code generation that separate format failure, silent financial error, and library uptake, and measure what enforced checks change: fewer wrong programs accepted, not more correct programs written. HiSTrim and persistent memory provide optional context and experience services; their auxiliary studies do not determine the core contribution.

2 Related work

FinAgent combines multimodal financial information with trading tools, reflection, and memory (Zhang et al., 2024); StockBench evaluates sequential portfolio decisions using prices, fundamentals, and news (Chen et al., 2025). The Finance Agent Benchmark evaluates financial research tasks (Bigeard et al., 2025). These works study tool use and memory within language-model agents. Our contribution supplies the financial knowledge, numerical contracts, and executable research checks that such agents can reuse. A separate exploratory comparison varies algorithm access under identical data.

Behavioral testing offers a structured view of model capabilities beyond aggregate scores (Ribeiro et al., 2020). Financial backtest overfitting and selection bias (Bailey and López de Prado, 2014; Bailey et al., 2017), alongside look-ahead leakage in financial agents (Li et al., 2025), motivate explicit checks on decision evidence. The persistent memory component adapts the mushroom-body model of Huang et al. (2024) to update state from delayed financial returns, linking within-task attention dynamics with cross-episode retention.

3 Library architecture: knowledge, computation, and verification

FIN-SKILLS connects three reusable layers. Skills describe financial assumptions and library conventions. Numerical interfaces execute methods against declared inputs. Research guards return structured findings that a controller can require before accepting an artifact. A successful algorithm call alone does not imply that the required audit ran.

Guarded research workflow. For a task and its available data, an agent selects a skill, inspects a numerical interface, and proposes an artifact. A controller executes the relevant checks on that artifact. Failures return actionable findings; missing required evidence remains incomplete. The controller accepts only artifacts meeting its declared policy and saves the associated execution records. This workflow is distinct from offering an agent optional tool access and trusting its claim to have validated the output.

Optional experience services. HiSTrim operates within a single session by evicting tokens from the KV cache during generation. Persistent memory operates across sessions by storing parameter updates in an external module. The two mechanisms operate on different timescales: context pruning controls what the model attends to right now, whereas persistent memory retains summary statistics of what happened in previous episodes. Neither service is required to run the library’s guards or reproduce its numerical primitives.

Grounded feedback updates. When an episode concludes, the agent records the market context, the chosen tool, and the realized return. Only verified, settlement-cleared outcomes update memory. Negative returns are retained as learning signals, while trades relying on look-ahead data or unadjusted prices are rejected by guards to prevent memory corruption.

4 Professional tools and domain knowledge

FIN-SKILLS exposes financial methods alongside the information an agent needs to run them: task scopes, input requirements, dependencies, parameter conventions, and dated source verifications. Domain skills route requests to relevant libraries;

Component	Role in research workflow	Target question	Implementation and validation
Financial skills	Route tasks and document dated conventions	What assumptions and failure modes apply?	129 skill documents
Numerical interfaces	Discover and execute quantitative methods	Do selected computations agree with references?	48 catalog entries; E3 tests eight primitives
Executable guards	Diagnose data, accounting, and validation defects	Which required checks passed, failed, or lack inputs?	36 registered guards; E1 exercises 13
Execution records	Bind verification to the submitted artifacts	Did the required checks actually run on this code and data?	Explicit pass, fail, and incomplete states
Optional extensions	Select context and retain verified experience	How do auxiliary context and memory services affect tasks?	HiSTrim and persistent memory; separate studies

Table 1: Core financial library components and optional context and memory integrations.

per-library skills provide implementation guidance. The algorithm catalog covers portfolio construction, forecasting, volatility estimation, technical signals, and execution schedules, providing uniform programmatic contracts over quantitative methods.

Agent-selected execution. An agent can discover candidate algorithms, inspect their requirements, and execute them with custom parameters. Structured outputs separate numerical results from missing dependencies or runtime errors. An optional strategy adviser provides rule-based suggestions without overriding model choices. In our evaluation, agents access 18 price-compatible algorithms, with requests and executions logged independently from free-form text.

Information and model interfaces. Retrieval interfaces chunk packaged skills and documents, returning relevant passages under strict as-of cut-offs to prevent look-ahead bias. Forecasting and prediction adapters provide a unified fit–predict lifecycle, while portfolio optimizers expose specialized execution interfaces. Appendix B details the complete interface inventory, model lifecycle, and retrieval pipeline.

5 Execution and feedback verification

Tool execution and memory updates require verifiable records of what actually ran. The execution controller computes cryptographic hashes over input data, executed code, and returned allocations. If an agent modifies an artifact after execution, previous validation receipts become invalid. The controller returns `PASS` if all required guards pass, `FAIL` if a guard detects a defect, and `INCOMPLETE` if evidence is missing. This mechanism enforces execution provenance without modifying the underlying algorithm.

For cross-episode memory, timing and accounting checks verify that delayed outcomes cleared settlement before they update persistent state. Trades that rely on look-ahead data or unadjusted corporate splits are blocked, preventing corrupted feedback from distorting the agent’s memory. Unsettled trades wait for settlement, while valid unprofitable trades remain eligible learning signals.

The audit study evaluates models under unconstrained generation, optional self-checking, and mandatory verification. Appendix A describes the evaluation setup, defect detection benchmarks, and empirical audit results across model tiers.

6 Library evaluation: defects, execution, and numerical parity

We evaluate the financial library at three boundaries: detecting invalid research inputs, executing declared interfaces, and reproducing numerical references; Section 7 adds the code-generation audit (E4). Table 2 summarizes the observed evidence. Each row has its own unit and denominator.

E1: detecting research defects. The benchmark plants 12 defect families, including temporal leakage, unadjusted corporate actions, and universe-selection errors, in three synthetic worlds. The suite records 36/36 detections and no false alarms on its clean references. This is suite-level recall: a defect may be caught by another applicable guard even when one intended check misses it. Repeating a family across seeds does not create a new defect type. Appendix A describes the independent oracle and the distinction between diagnosis and agent repair.

E2: execution and audit compliance. We count actual dispatches and final artifacts rather than statements that a check ran. The exploratory

Study	Endpoint	Observed result	Scope
E1: guards	Planted-defect detection	36/36 detected (100%); no clean-reference false alarms	12 defect families in three seeded worlds; 13 core guards exercised
E2: execution	Valid decisions and successful tool requests	64/64 valid decisions across four conditions; 43 successful autonomous tool requests	16 autonomous decisions; requests include discovery, reading, advice, and computation
E2: audit	Artifact validation and reporting fidelity	Reference submission accepted; pilot Opus Sharpe gap 0.188 to 0.034	Historical pilot; self-reported guard use is not an execution receipt
E3: parity	Agreement with numerical references	8/8 checks within case-specific tolerances	Selected primitives; maximum absolute error 1.18×10^{-8}
E4: code	Valid programs; accepted but wrong	First submission 0/48 (1/48 post hoc). Enforced, three tries: valid 1, 1, 0 of 24; accepted but wrong 22, 2, 0	Two model sizes, four rules, three seeds; arms: raw, raw with checks, library with checks

Table 2: Core library evidence. Detection, executable completion, audit acceptance, and numerical agreement are different endpoints. No row establishes universal correctness or profitable trading.

portfolio study completed all 64 decisions, including 16 with autonomous tool access, and logged 43 successful requests in that arm. These requests include catalog lookup and guidance retrieval as well as numerical algorithms. In the separate audit feasibility study, optional guards and mandatory enforcement have different acceptance outcomes. Mandatory rejection protects the submission boundary but does not constitute successful repair: the 14B model reached its turn limit in all three mandatory cells. The reference submission passed the public checks and independent grading.

E3: numerical parity. Eight checks compare equal weighting, inverse-volatility weighting, minimum-variance optimization, normal and historical VaR/ES, historical volatility, fractional-difference weights, and an augmented Dickey-Fuller statistic with closed forms or independent NumPy, SciPy, pandas, and statsmodels references. All meet their recorded tolerances; the largest absolute difference is 1.18×10^{-8} for the optimization weights. The optimization case allows 10^{-3} while other cases have their own tighter tolerances. This supports agreement for the tested calculations, not exact equality or validation of all catalog entries.

Auxiliary integrations. The context and memory studies test optional consumers of the library’s results. Their methods and outcomes appear in the appendices. The public FinQA RAG and reranking runs retain their original official scores (4/192 correct programs across six model and retrieval cells). A post-hoc reanalysis bound its target-blind extractions by hash before grading: accepting a single fenced answer adds no correct pro-

gram, and counting explicit correct final numbers raises the total to 7/192. That second count is not program accuracy. Of the 96 Mistral answers, 75 have no final answer, so format repair alone cannot isolate retrieval quality (Appendix B). Interface corrections require a new versioned comparison rather than retrospective replacement of the recorded grades.

7 E4: Financial code generation with and without enforcement

Can an open code model write a correct allocation or signal rule from a specification alone, and does the library’s documentation, or enforcing its use and checks, change that? We froze a paired first-submission study before model inference. Qwen2.5-Coder 7B and 14B each wrote one program for four rules (diagonal risk parity by inverse volatility, positive cross-sectional momentum, a 5/20 moving-average crossover, and volatility-targeted momentum), three data seeds, and two arms: 48 programs. Both arms received identical raw quotes with split jumps, cumulative split factors known on their event dates, the mathematical rule, the one-bar timing rule, one response of at most 2,048 tokens, and no execution feedback. The library arm additionally received the actual catalog output and interface and guard documentation (mean prompt 11,667 versus 10,756 tokens). Programs ran in the Landlock/seccomp worker. An independent NumPy grader, shared by both arms, requires execution, agreement with the reference within 10^{-8} , unchanged positions through session k when prices at and after k are perturbed ($k = 32, 64$), and identical output for

the economically equivalent split-adjusted input. A program is valid only if it passes all.

Pre-registered outcome. No program was valid in either arm (0/48). The protocol asked for Python only, optionally inside one fence. In all, 37 responses appended an explanation after the fence (24 from 7B and 13 from 14B), so the frozen parser compiled prose and recorded a `SyntaxError`. The 11 executed programs all failed numerical agreement and split invariance. No response was truncated. None of the 24 library-arm responses imported or mentioned the library, and no algorithm or guard call was traced. The comparison therefore measures documentation exposure without uptake; it does not estimate the effect of using FIN-SKILLS.

Post-hoc extraction and failure modes. To separate formatting from computation, we froze an arm-blind extraction of the first complete fence by hash before execution and reran the same worker, sandbox, and grader on another Linux host; it reproduced all 11 official executions exactly. Then 37 programs execute, but only 1 of 48 is valid. The failures are silent in the sense that matters for research: 24 of 24 14B programs, in both arms, applied a cumulative product to split factors that the prompt defines as already cumulative, and 26 executed programs change their positions when the same prices are presented split-adjusted. In 15 executed programs a position moves when prices at or after its own session change; 12 use the same-session close and 3 use later data. The 7B programs that still fail raise shape, index, or pandas-API errors.

What the library guard observed. Run post hoc on each executed program, the library’s causality guard flagged all 3 programs with strictly-future dependence and none of the 22 programs that passed the grader’s timing probes. It passed the 12 same-session violations: the guard asks whether values before k depend on data at or after k , which is feature causality, whereas a position must also exclude session k itself. The execution-lag contract therefore needs its own check rather than an inference from a passed causality test. Together with E2, these results show that raw first submissions are unreliable in ways that still execute, and that optional documentation is not used.

Enforced library calls and checks. We therefore froze a follow-up on the same rules, seeds, models, prompt packet, and hidden grader, with up to three attempts and controller feedback in three arms of 24 episodes: coding from scratch with execution feedback only; coding from scratch with enforced lag and split checks; and the library documentation with an enforced `fin_skills.algorithms.run` call plus the same checks. The lag check applies the library’s causality guard to positions shifted up one row ($k = 40, 72$), closing the same-session gap above; the split check requires identical output for the split-adjusted representation. Feedback never reveals reference weights, and every arm parses the first complete fence. A deliverable counts only if the controller accepts it and the unchanged grader finds it valid.

Enforcement did not produce more valid programs (Table 4): 1, 1, and 0 of 24. It changed what is delivered. With execution feedback alone, 23 programs were accepted and 22 of them were wrong (91.7%), including 12 split and 9 timing failures. With enforced checks, accepted-but-wrong programs fell to 2 (8.3%), both of which pass the grader’s timing and split probes and fail only numerical agreement; with the library requirement, none was accepted. The library was hard to use from its interface text: 42 of 72 library-arm attempts reference FIN-SKILLS, 8 trace a call, and 51 fail to run, mostly on output shape, nonfinite values, and argument errors. In 7 attempts the model followed a parameter name in the supplied interface text that the function does not accept; all 7 also re-cumulated split factors and would have failed the split check. For these models and this budget, the checks turn silent errors into explicit rejections; correct construction still needs a stronger model, more repair budget, or a simpler interface.

8 Component ablation: algorithm access without mandatory guards

This exploratory study tests a restricted algorithm-access component of the library. The dispatcher offers discovery, skill reading, computation, and optional advice, but does not enforce the library’s research guards on final allocations. We treat it as an unguarded component ablation, not as a test of the complete guarded workflow. No matched guarded arm was run in this experiment.

Model	Arm	Executed		Post-hoc extraction				Mean per response	
		Official	Post-hoc	Correct	Split fail	Timing fail	Valid	Output tok.	Gen. s
7B	Raw	0/12	7/12	1	2	2	1/12	398	8.4
7B	Library	0/12	6/12	0	0	4	0/12	337	7.8
14B	Raw	5/12	12/12	0	12	6	0/12	312	9.4
14B	Library	6/12	12/12	0	12	3	0/12	286	9.0

Table 3: Paired first-submission code generation. Under the pre-registered parser no program is valid in any cell (0/48). Post-hoc columns take the first complete code fence; failure counts are among executed programs. No library-arm program called a library algorithm or guard.

Arm	Valid	Acc.	Wrong	Lib.	Tries
Raw	1/24	23	22	—	1.25
Raw + checks	1/24	3	2	—	2.79
Lib. + checks	0/24	0	0	5/24	3.00

Table 4: Enforced follow-up (job 1765432; 7B and 14B pooled, up to three attempts). Raw receives execution feedback only. Valid: accepted and valid under the hidden grader. Acc.: accepted. Wrong: accepted but invalid. Lib.: episodes with a traced library call in any attempt. Tries: mean attempts.

8.1 Experimental setup

We test Qwen2.5-Coder 7B and 14B under four conditions: historical prices alone; prices with static skill excerpts; prices with precomputed tool outputs; and autonomous access to the library’s algorithm catalog. Each condition runs across inference seeds 11 and 23, with a budget of five interaction turns per decision (capped at 256 generated tokens per response). Conditions run in randomized order.

In the autonomous condition, the agent receives descriptions of available tools. It can inspect algorithm specifications, run an algorithm with custom arguments, or query a rule-based strategy adviser. Tool calls are dispatched against trailing market snapshots and return structured numerical results. In our evaluation, agents access 18 price-compatible quantitative routines.

A decision without valid portfolio weights leaves existing allocations unchanged. Requests, outputs, and intermediate reasoning traces are logged separately, and final weights are frozen before scoring.

8.2 Data and evaluation protocol

We evaluate on daily exchange rates for EUR against USD, JPY, GBP, and CHF from ECB reference data. Messages use anonymized asset identifiers and relative indices rather than calendar

Condition	7B	14B
No library	+0.100	−3.324
Skill text	−2.930	−4.567
Fixed tool outputs	−3.075	−3.030
Autonomous library	−3.984	−3.783

Table 5: Mean cumulative proxy return (%) across inference seeds with 5 bps transaction costs per side.

dates.

Each decision observes 126 trailing days. The evaluation spans the first 84 trading days of 2025 (ending May 2), with four rebalancing points spaced 21 days apart. Decisions at step t execute at day $t + 1$ prices to prevent look-ahead bias. Portfolios are long-only without leverage, with unallocated capital held in cash. We charge 5 basis points per side for rebalancing and final liquidation. Cash, equal-weight buy-and-hold, and deterministic rules serve as reference baselines.

8.3 Observed execution and returns

Across 16 trials, models generated 64 valid final portfolio allocations. The autonomous conditions account for 16 decisions and 43 successful requests: 13 from 7B and 30 from 14B. Requests include catalog discovery, skill reading, strategy advice, and algorithm execution; they are not 43 numerical algorithm executions. The completed autonomous conditions recorded no tool or format errors. An earlier parser-failure attempt is retained separately and is not included in these completed trials.

Tool choices diverged across model scales: 7B selected cross-sectional momentum, Bollinger reversion, and volatility-targeted momentum, whereas 14B predominantly used Bollinger reversion alongside trend-following indicators.

Table 5 shows returns across all arms. While autonomous 14B improved over its text-only condition by 0.783 percentage points, neither autonomous model beat the unassisted baseline

(+0.100% for 7B, −3.324% for 14B).

Recorded portfolio holdings show different exposure choices across conditions: mean risky asset exposure for 7B rose from 0.500 without the library to 0.969 with autonomous tools; for 14B, exposure shifted from 1.000 (all-in) to 0.500. Autonomous runs used 38 model calls per model across eight decisions, totaling 1,155,782 prompt tokens and 10,165 completion tokens.

The valid decisions and successful requests provide interface evidence for E2. The returns describe one short, anonymized currency path with transaction costs. They do not establish that the financial library is ineffective, nor that missing guard enforcement caused the losses: the experiment lacks the paired intervention required for either conclusion. A full-workflow comparison must hold data, models, budgets, execution rules, and costs fixed while varying mandatory guards. Memory and context pruning are separate interventions and are not inferred to remedy these returns.

9 Discussion: financial reliability as an executable contract

The strongest evidence for FIN-SKILLS concerns defects that its checks can observe and computations that can be independently reproduced. E1 establishes detection on the fixed planted families; E3 establishes numerical agreement for the tested primitives. E2 shows why optional access and enforced verification must be distinguished: an agent may describe a check without executing it, while mandatory verification may reject a submission that the agent cannot repair within its budget. Detection, acceptance, and task success must therefore be reported separately.

E4 sharpens both points. Unassisted first submissions from open code models failed in the ways a backtest would not reveal: split conventions, the execution lag, and output contracts. Supplying documentation is not the same as supplying the library, because no program used it, and a guard certifies only its stated contract: the causality check caught every strictly-future dependence but not a same-session lag until it was composed with a one-row shift. Enforced, these checks reduced accepted-but-wrong programs from 91.7% to 8.3% of episodes, yet no arm produced more valid programs, and enforced library use mostly failed on interface errors. The library’s mea-

sured value here is refusing wrong results; making agents use its algorithms correctly requires a clearer calling interface and stronger models.

The portfolio component ablation leaves the causal contribution of guard enforcement unmeasured. Exposure choices, market direction, and transaction costs all affect its returns; the saved losses do not identify which factor caused the difference between conditions. A matched guarded arm is needed to measure what enforcement changes.

Context pruning and persistent memory can consume the library’s verified records, but their gains are separate empirical questions. In the auxiliary report-QA studies, a post-hoc reanalysis keeps the official program scores and shows that relaxing fence parsing adds no correct program; missing final answers, not formatting alone, limit those runs. The current context-pruning study shows quality loss. These findings motivate interface and retention checks, not a replacement of the library’s contribution with an unproven memory benefit.

10 Conclusion

FIN-SKILLS makes professional financial guidance executable through quantitative interfaces and research guards. Its fixed defect benchmark records 36/36 detections with no clean-reference false alarms, and 8/8 numerical checks pass their stated tolerances. A frozen code-generation audit finds 0/48 valid first submissions under its pre-registered parser and 1/48 after post-hoc extraction, with silent split and timing errors dominating the executable programs. Enforcing the library’s checks cut accepted-but-wrong programs from 22 of 24 to 2 without raising valid ones. These results support reusable infrastructure that refuses spurious research findings and reproduces computations; they do not show that the library makes small models write correct financial code.

Limitations

The defect benchmark repeats fixed synthetic families across seeds; its 100% recall is not a universal leakage-detection guarantee. Numerical parity covers eight primitives, not every backend. The code-generation study uses author-designed synthetic tasks, two model sizes from one family, and three seeds on four fixed rules, which are not twelve independently sampled problems; training

contamination is not ruled out. Its fence extraction is a post-hoc sensitivity analysis run on another host. The enforced follow-up reuses these tasks after seeing E4’s failures, allows only three attempts, and its interface text named one argument differently from the library function. Passing sampled perturbations does not certify universal causality. The currency component ablation uses daily reference prices, two Qwen model sizes, and one short path, without a matched mandatory-guard arm or executable quotes and carry. Auxiliary QA comparisons use public, development-exposed, overlapping tasks and retain format and runtime failures; their format reanalysis is post hoc. Research guards enforce specified checks, not regulatory certification, complete adversarial isolation, or investment profitability. Human workflow efficiency and independent prospective validation remain unmeasured.

Ethics and Reproducibility

All models, tools, and evaluation benchmarks adhere to open, reproducible protocols. Software environments, execution digests, and audit traces are publicly available to facilitate independent verification.

References

- David H Bailey, Jonathan Borwein, Marcos López de Prado, and Qiji Jim Zhu. 2017. The probability of backtest overfitting. *Journal of Computational Finance*, 20(4):39–69.
- David H Bailey and Marcos López de Prado. 2014. The deflated Sharpe ratio: Correcting for selection bias, backtest overfitting, and non-normality. *The Journal of Portfolio Management*, 40(5):94–107.
- Antoine Bigeard, Langston Nashold, Rayan Krishnan, and Shirley Wu. 2025. Finance agent benchmark: Benchmarking llms on real-world financial research tasks. *arXiv preprint arXiv:2508.00828*.
- Yanxu Chen, Zijun Yao, Yantao Liu, Amy Xin, Jin Ye, Jianing Yu, Lei Hou, and Juanzi Li. 2025. StockBench: Can LLM agents trade stocks profitably in real-world markets? *arXiv preprint arXiv:2510.02209*.
- Cheng Huang, Junjie Luo, Seung Je Woo, Lucas A. Roitman, Jizhou Li, Vincent A. Pieribone, Madhuvanthi Kannan, Ganesh Vasan, and Mark J. Schnitzer. 2024. Dopamine-mediated interactions between short- and long-term memory dynamics. *Nature*, 634:1141–1149.
- Xiangyu Li, Yawen Zeng, Xiaofen Xing, Jin Xu, and Xiangmin Xu. 2025. Profit mirage: Revisiting information leakage in llm-based financial agents. *arXiv preprint arXiv:2510.07920*.
- Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. 2020. Beyond accuracy: Behavioral testing of NLP models with Check-List. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4902–4912.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*.
- Wentao Zhang, Lingxuan Zhao, Haochong Xia, Shuo Sun, Jiaze Sun, Molei Qin, Xinyi Li, Yuqing Zhao, Yilei Zhao, Xinyu Cai, Longtao Zheng, Xinrun Wang, and Bo An. 2024. A multi-modal foundation agent for financial trading: Tool-augmented, diversified, and generalist. *arXiv preprint arXiv:2402.18485*.

A Supporting audit and validation studies

This appendix details the execution and feedback validation checks, defect recall benchmarks, and empirical audits across model tiers.

A.1 Independent evaluation environment

We evaluate agent portfolio decisions in a synthetic market environment with full ground-truth transparency, providing quoted prices, corporate split actions, delisting events, and fundamental disclosures. A submitted strategy script returns portfolio weights, alongside a written report stating its expected Sharpe ratio. Session- t portfolio weights must use only information available by the close of session $t - 1$.

An independent oracle evaluator verifies submissions by perturbing future data, checking for post-delisting exposure, and recalculating performance under fixed transaction costs. Oracle grades are generated after the submission receipt is saved and are never exposed to the agent during its decision turns.

The oracle tests whether allocations depend on unavailable information. The recorded latent-drift reference earns a sample net Sharpe of 1.63 with a one-day lag; this is a reference strategy, not a proved ceiling on attainable performance. Wrong-side joins can inflate reported performance, but a high Sharpe alone does not diagnose leakage. Dependency perturbations and availability-time checks establish the defect.

A.2 Validation benchmarks

Defect detection benchmark. We evaluate 13 core research guards against 12 planted quantitative defect families, including look-ahead feature joins, unadjusted stock split prices, future earnings disclosure leaks, and survivorship bias. Across three seeded synthetic market worlds, the guards achieve 100% recall (36/36 planted defects caught) with zero false alarms on clean reference code.

Reporting discrepancy and compliance paradox. Table 6 measures reporting discrepancy: the absolute difference between the Sharpe ratio claimed by the agent in its written report and the ground-truth Sharpe ratio computed by the oracle evaluator.

Without tools, Opus exhibited a reporting gap of 0.188, which decreased to 0.034 when given

Model and condition	Mean absolute Sharpe gap
Opus, no library	0.188
Opus, with library	0.034
Haiku, no library	0.574
Haiku, with library	11.949

Table 6: Reporting discrepancy between agent-claimed Sharpe ratios and ground-truth oracle evaluations.

execution guards (81.9% reduction). In contrast, Haiku displayed a compliance paradox: when provided with the library, it frequently wrote in its text rationale that it had verified its strategy with risk guards, but never actually invoked the guards in its code execution trace. This superficial compliance inflated its discrepancy to 11.949, demonstrating that smaller models require automated code execution verification rather than relying on self-reported compliance.

Social sentiment verification. On 1.72 million financial posts across 2,521 social media accounts, raw uncalibrated sentiment scores exhibited negative out-of-sample rank correlation with subsequent asset returns (Rank IC = -0.0318). By evaluating each account’s historical prediction accuracy and applying Bayesian credibility weighting (including inverting signals from consistently inaccurate accounts), the out-of-sample correlation was reported as $+0.0104$. These values are retained from the supplied summary fixture; row-level predictions and timestamp provenance were unavailable for independent reproduction. They are not verified trading-return results or core library evidence.

A.3 Four-condition audit feasibility

We evaluated models under four audit conditions: unassisted baseline (C0), text guidance only (C1), optional executable guards (C2), and mandatory automated audit (C3).

Operational findings. Table 7 presents results for Qwen2.5-Coder 14B under automated audit constraints. The acceptance rate was 67% under C0 and C1, 100% under C2 (optional guards), and 0% under C3 (mandatory audit), where runs reached the turn limit while attempting to fix all detected code defects. Mean token consumption under C3 was 47,491 compared to 13,877 under C2. This records additional work without successful acceptance; token consumption alone does not establish better debugging.

Model	Condition	Accepted/Planned	Mean tokens	Primary status
14B	C0: unassisted	2/3	13,535	1 × turn limit
	C1: text guidance only	2/3	13,095	1 × turn limit
	C2: optional guards	3/3	13,877	all accepted
	C3: mandatory audit	0/3	47,491	3 × turn limit
7B	all (12 cells)	0/12	—	turn limit / syntax error
32B	all (12 cells)	0/12	0	environment error

Table 7: Audit feasibility results across open-weight models under automated verification constraints.

A reference quantitative script passed all verification and timing checks on the market benchmark, confirming the end-to-end viability of the automated verification pipeline.

Component	Entries
Skill documents	129
Algorithm-backend catalog	48
Registered research guards	36
JSON-callable tools	56

Table 8: Inventory of quantitative research library interfaces.

B Library implementation and interface scope

Knowledge and executable interfaces. Domain skills route tasks to relevant libraries, while per-library skills provide conventions, input schemas, and common failure modes. The Python package exposes catalog search, skill loading, and reference retrieval. Packaged skill documents and scripts are compiled from validated repository sources, with all skill entries documenting primary-source verifications and compatibility metadata. Table 8 summarizes the core library components.

Algorithm selection and execution. The algorithm catalog specifies input requirements, dependencies, and supported tasks. Implemented families include portfolio construction, forecasting, volatility and risk estimation, technical signals, and execution schedules. Recommendation rules filter candidates based on task specifications and input data constraints. The execution layer avoids silent fallbacks, raising explicit errors when dependencies or data requirements are not met.

Reusable models and lifecycle. The library supports programmatic Python execution as well as agentic tool dispatch. Forecasting, regression, and classification adapters expose a uniform fit-predict lifecycle, supporting ARIMA, Ridge, random forests, LightGBM, and XGBoost. Fitted models and estimators can be serialized and restored alongside environment metadata. The model registry additionally provides an adapter for the bio-inspired memory architecture via `create_model("fly_memory")`, supporting prediction, verified feedback updating, and state serialization.

Built-in retrieval and model composition. The RAG interface chunks packaged skills or external documents and retrieves passages using BM25 and neural rerankers. Passages retain document identifiers, source offsets, and availability times-

tamps; point-in-time retrieval excludes documents unavailable at the requested decision cutoff. Citation checks validate references against retrieved passage identifiers. The pipeline freezes candidates, temporal cutoffs, and context budgets prior to neural reranking.

Information availability and point-in-time processing. News and disclosure collection supports RSS/Atom feeds, static pages, GDELT, and official regulatory sources. Collection records distinguish publication time from observation time and retain revision histories. Point-in-time digests filter future, stale, and undated records while deduplicating canonical URLs.

Model choice and rule-based advice. An agent can inspect skills, discover candidate algorithms, execute chosen methods with custom parameters, and incorporate numerical outputs into its decision logic. Market-state categorization provides heuristic regime indicators (trend, range, stress) that agents can inspect as additional evidence alongside price tables.

Research workflow and audit boundaries. Research interfaces enforce chronological validation splits, out-of-sample holdout reporting, and standardized trial records. Python and JSON tool interfaces return structured dictionary outputs, ensuring that all intermediate calculations can be logged and audited.

Local decision models. Adapters for Kev¹ and Laya² expose structured decisions and passage reranking from local checkpoints, without hosted API credentials. Real GPU integration checks covered Kev-0.8B, Kev-4B, and the English and multilingual Laya checkpoints, including RAG composition and rejection of oversized inputs. Original responses and checkpoint identities are retained; bounded normalization of rounded probabilities does not establish calibration.

Exploratory routing regression. We froze the same three BM25 skill candidates for each of 108 public regression queries and evaluated a seeded shuffled order and its reversal. BM25 selected the expected skill on 76 queries. Kev-4B selected it on 79 and 82 queries, respectively, but changed its decision on 24 queries when the order changed. Kev-0.8B obtained 45 and 50 correct selections.

¹<https://github.com/jaredpalmer/kev>

²<https://github.com/NandhaKishorM/laya>

Both Laya variants were rejected before inference on all queries because their option budgets would truncate the full skill descriptions; these rejections do not estimate prediction accuracy. The fixtures were previously used in development and contain only two Chinese queries. This comparison measures public routing regression, not unseen financial-task performance or successful tool execution. BM25 remains the default, with Kev-4B available as an optional decision step.

Storage recovery and revision ordering. In a separate engineering study, we terminated collector processes before saving, during a batch, immediately before transaction commit, and after commit returned, with three repeats per boundary. All 12 cases recovered the expected atomic state across event records, latest-version pointers, pending alerts, and the collection cursor; independent read-only database checks confirmed these states. Runs with 100, 1,000, and 10,000 synthetic events retained the expected record counts and cursor. These checks cover process termination, not machine power loss or financial task accuracy. An out-of-order revision test also exposed an interface boundary: the storage layer’s latest pointer follows arrival order, so an older revision arriving last becomes current. Historical research must therefore apply an explicit chronological or availability-time selection policy rather than infer publication order from that pointer.

Report-QA workflow ablation. On 32 public FinQA questions from 16 companies, we compared BM25S components, the same retrieval with a frozen financial skill, the library RAG interface with that skill, and all supplied report text and tables. Qwen2.5-Coder-14B-Instruct produced 3, 2, 2, and 1 correct answers, respectively; Mistral-Nemo-Instruct-2407 produced none in these conditions. Each model generated 128 answers. Applying the citation-label gate to the same frozen RAG answers changed no outcomes; this paired check adds no independent samples and does not test semantic support. For Qwen, incorrect accepted answers numbered 5, 1, 0, and 5, respectively. The smaller number of wrong acceptances therefore accompanies low correct completion and frequent rejection or format failure, rather than demonstrating an overall accuracy improvement. The full-context condition uses FinQA’s extracted text and tables, not complete original annual reports.

Answer model	Reranker	Correct	Errors
Qwen	BGE	2/32	30/32
Qwen	Kev-4B	0/32	31/32
Mistral	BGE	0/32	32/32
Mistral	Kev-4B	0/32	32/32

Table 9: Exploratory FinQA reranking transfer. Errors count saved scoring or answer-format exceptions, not failed compute jobs. One Qwen/Kev answer was incorrect without such an exception. Model identities are given in the text.

Model	Condition	Official	Fence	Recov.	No final
Qwen	RAG	2	2	3	3
Qwen	BGE	2	2	2	3
Qwen	Kev-4B	0	0	2	3
Mistral	RAG	0	0	0	26
Mistral	BGE	0	0	0	24
Mistral	Kev-4B	0	0	0	25

Table 10: Post-hoc FinQA format reanalysis (32 questions per row). Official: original strict program correctness. Fence: single-fence parser. Recov.: recoverable correct final answer, which is not program or citation correctness.

Downstream transfer from passage reranking.

Using the same 32 questions, we froze the ten BM25 passage candidates and reranked them with BGE or local Kev-4B before supplying up to five passages within the same context budget. The answer agent, financial skill, calculator, and response limits were held fixed. All 64 planned answers per model were saved. Table 9 reports their original scores, which were independently replayed against the official numerical and program labels. These questions overlap the workflow and memory studies; the 128 generated answers are not 128 independent tasks.

Frozen format reanalysis. To test whether answer formatting alone explains these low scores, we re-scored the 192 saved RAG and reranking answers without regenerating any of them. Target-blind extractions were bound by hash before the official FinQA executor and gold labels were loaded, and the original official counts were reproduced first. Table 10 keeps those program scores, adds the prospective single-fence parser, and separately counts a final answer as recoverable when the parsed program is correct or the final text states one unambiguous correct number. Percent signs are normalized; refusals, ambiguous scales, and missing answers stay failures, and no tool output replaces a missing final answer.

Relaxing the fence adds no correct program. Recoverable final answers exceed program correctness in 3 answers (3 from Qwen), and most Mistral answers never reach a final answer. Formatting is therefore one failure among several, not the explanation for the scores, and the reanalysis does not establish a retrieval or reranking gain.

Interpretation and runtime limitation. The reranking study does not demonstrate a Kev downstream-answer advantage. Answer-format and scoring exceptions dominate these runs. The original Mistral runtime also emitted a tokenizer-regex warning; the frozen run was not modified after inspecting its outcomes. Consequently, its zero correct counts do not isolate model capability from tokenization and answer-format problems. The audit verifies saved outputs, tool receipts, and grades, not the validity of every runtime choice. These public, overlapping tasks do not establish unseen-task generalization, semantic citation support, or human development-time savings.

C Within-task context selection with HiSTrim

Financial agents frequently execute quantitative tools that return verbose tables of historical prices, factor loadings, and optimization logs. Over multi-turn interactions, these intermediate outputs quickly fill the model’s context window and slow down generation. Within-task context selection determines which interaction records should remain in active attention and which can be pruned.

HiSTrim prunes intermediate interaction turns while strictly preserving task instructions, problem constraints, and asset identifiers.

C.1 Protected, sign-preserving routing

We partition the context into two sets: protected tokens $\mathcal{P}$ (containing system prompts, task instructions, and the current query) that are never evicted, and historical interaction turns $\mathcal{H}_{\ell-1}$ (such as past tool outputs and intermediate code traces) eligible for pruning at layer ℓ .

For each interaction turn representation h_m , a dual-head router computes a gating logit g_m and a utility logit u_m :

$$g_m = w_g^\top h_m + b_g, \quad u_m = w_u^\top h_m + b_u, \quad (1)$$

$$r_m = \sigma(g_m)u_m, \quad \sigma(g) = (1 + e^{-g})^{-1}. \quad (2)$$

Here $\sigma(g_m) \in (0, 1)$ modulates the utility magnitude. Because $\sigma(g_m) > 0$, $\text{sign}(r_m) = \text{sign}(u_m)$. Using the sigmoid gate ensures that positive utility logits always produce positive routing scores, avoiding the sign reversals that occur with activation functions like SiLU when both logits are negative.

Scores are standardized across the sequence and filtered with a learned threshold τ_ℓ :

$$z_m = (r_m - \mu_\ell) / (s_\ell + \epsilon), \quad (3)$$

$$\mathcal{H}_\ell = \{m \in \mathcal{H}_{\ell-1} : z_m \geq \tau_\ell\}, \quad (4)$$

$$\mathcal{T}_\ell = \mathcal{P} \cup \bigcup_{m \in \mathcal{H}_\ell} \mathcal{I}_m, \quad (5)$$

where $\mathcal{I}_m$ denotes the token positions belonging to turn m . This guarantees that protected instruction tokens $\mathcal{P}$ remain in the active set $\mathcal{T}_\ell$ at all layers.

C.2 Forward-only calibration and buffer compaction

Because fine-tuning the full LLM backbone on every task is computationally expensive, the router

direction w is calibrated using forward-only finite differences on the unit sphere:

$$w_0 = \frac{\sum_s \alpha_s v_s / \|v_s\|_2}{\|\sum_s \alpha_s v_s / \|v_s\|_2\|_2}. \quad (6)$$

Weights are updated in the tangent space $g_\perp = (I - ww^\top)g$ followed by normalization, allowing the router to adapt without full backpropagation through the LLM.

At inference time, rather than retaining evicted tokens with an attention mask, HiSTrim physically gathers surviving entries into a contiguous packed KV-cache buffer. This reduces active memory bandwidth and accelerates token decoding.

C.3 Attention sink compensation

Transformer attention concentrates heavily on initial prefix tokens (attention sinks; Xiao et al., 2024). Evicting intermediate tokens reduces the softmax denominator, which can distort attention probabilities over surviving tokens. To maintain calibrated attention distributions, we add an offset to the sink logit s_0 :

$$\Delta^* = \log \left(1 + \sum_{j \in \mathcal{E}} e^{s_j - s_0} \right), \quad (7)$$

where $\mathcal{E}$ is the set of evicted token indices. In practice, the lightweight count heuristic $\beta \log(|\mathcal{E}| + 1)$ absorbs the removed probability mass without requiring full exponent recomputation.

Empirical performance. On sequential recommendation tasks (Appendix E), HiSTrim achieves up to $2.82\times$ inference speedups while retaining under 50% of historical KV tokens, without degrading generation quality.

D Cross-episode experience and persistent memory

Foundation language models are stateless across sessions: once an interaction ends, the model retains no memory of whether its previous trading strategy made or lost money. Fine-tuning the entire LLM with gradient descent after each trading day is impractical, computationally expensive, and prone to overfitting on noisy financial markets.

To provide cross-episode memory without altering LLM weights, we use an external associative memory module inspired by the fruit-fly mushroom body (Huang et al., 2024). In biological systems, the mushroom body maps complex

sensory features into sparse representations and uses dopamine-gated synaptic plasticity to associate sensory cues with delayed rewards and punishments across multiple decay rates (short- and long-term memory).

Associative memory architecture. Our memory module projects market state features and candidate actions into a high-dimensional sparse representation. When the agent evaluates a new trading decision, it queries the memory module for associative preference scores based on prior experiences in similar market regimes. These scores serve as contextual guidance alongside the prompt, without modifying the frozen LLM backbone.

Updating from complete holding episodes. Financial rewards are inherently delayed: an allocation chosen today only yields returns after a multi-day holding period. The feedback interface models an entry and its subsequent holding days as a single complete decision episode. For positive account values $W_0, \dots, W_T$, the target return is

$$G = \sum_{t=0}^{T-1} \log(W_{t+1}/W_t) = \log(W_T/W_0). \quad (8)$$

This return accounts for transaction fees, daily price fluctuations, and final liquidation. Memory updates fire only after the position is fully closed and settlement clears.

The verification interface checks timestamp ordering, return accounting, and data lineage. Trades relying on look-ahead data or unadjusted prices are rejected before reaching the memory module, preventing spurious profits from corrupting associative weights. Valid negative returns remain active learning signals, allowing the agent to learn from losses. Each verified trade receipt updates memory exactly once.

Isolating learning from risk filtering. To verify that performance gains reflect genuine associative learning rather than passive risk reduction, our benchmark compares the updating memory against a frozen-memory control and a volatility exposure gate under identical historical splits and costs. This setup isolates experiential adaptation from static volatility filtering.

E Empirical evaluation of selective context (HiSTrim)

This section presents empirical benchmarks evaluating HiSTrim across sequential decision and generative recommendation tasks, assessing its context routing fidelity, KV memory compression, and inference acceleration.

Generative sequential recommendation. Table 11 reports performance across five representative sequential tasks. Across both 1.7B and 8B parameter scales, HiSTrim maintains high retrieval and generation accuracy (e.g., MRR of 19.21 on News with 8B) while reducing the historical KV footprint to under 50%, substantially outperforming static pruning and dense baselines in efficiency.

Calibration-seed stability. Evaluating across five calibration partitions for OneRec-1.7B Ad yields Pass@32 of $18.78\% \pm 0.72\%$ and MRR@32 multiplied by 100 of 3.81 ± 0.19 (mean and sample standard deviation). This confirms that the learned router directions and standardized thresholds remain highly stable across different calibration partitions.

Task accuracy, agreement and throughput. Table 12 presents rating and ranking benchmarks under varying context budgets. On Goodreads with Qwen3-8B, HiSTrim achieves lower MAE (1.078 vs. 1.232) while retaining only 35% of historical KV cache. On the hybrid Amazon ranking task, HiSTrim achieves higher NDCG@10 (0.5567 vs. 0.5183) while providing a $1.78\times$ throughput speedup over the dense model, with the aggressive variant yielding a $2.82\times$ speedup at 35% KV retention.

Model	Method	Hist. KV (%)	Ad	Product	Video	Label-cond.	Interactive
1.7B	Full Dense	100.0	3.24	7.81	5.16	1.25	11.95
1.7B	TokenDrop-50	50.0	3.08	7.72	4.89	1.29	11.70
1.7B	H2O	54.6	3.98	7.51	7.77	1.25	11.72
1.7B	SnapKV	54.6	3.85	7.75	7.36	1.30	11.85
1.7B	Router-Tuning	45.7	4.00	7.27	3.30	1.18	11.24
1.7B	HiSTrim	47.5	4.10	8.09	6.43	1.50	12.08
8B	Full Dense	100.0	4.56	8.22	7.67	1.41	18.25
8B	TokenDrop-50	50.0	4.22	7.89	7.12	1.32	17.40
8B	H2O	54.6	5.06	8.28	8.94	1.37	18.95
8B	SnapKV	54.6	5.02	8.25	8.54	1.72	18.70
8B	Router-Tuning	46.2	4.12	7.95	6.84	1.34	17.65
8B	HiSTrim	49.5	5.25	8.54	7.96	1.47	19.21

Table 11: Generative sequential recommendation performance across tasks: $\text{MRR@32} (\times 100)$ evaluated over 1,000 users per task with 32 beams. Historical KV retention excludes protected tokens. HiSTrim matches or exceeds dense retrieval quality while reducing historical KV footprint by more than half.

Task / model	Method	Hist. KV (%)	Label metric	Teacher metric	Speedup
Beauty / Qwen3-0.6B	Dense	100.0	MAE 1.642	KL 0.000	—
	TokenDrop-50	50.0	MAE 1.517	KL 0.233	—
	HiSTrim	50.1	MAE 1.643	KL 0.084	—
Goodreads / Qwen3-8B	Dense	100.0	MAE 1.232	—	—
	HiSTrim	35.0	MAE 1.078	Teacher MAE 0.651	—
Amazon / DCNv2+LM	Dense	100.0	NDCG@10 0.5183	—	1.00×
	TokenDrop-50	50.0	NDCG@10 0.5204	—	1.79×
	HiSTrim	50.0	NDCG@10 0.5567	—	1.78×
	HiSTrim aggressive	35.0	NDCG@10 0.5065	—	2.82×

Table 12: Rating and ranking comparisons across sequential tasks. Lower MAE and KL are better; higher NDCG is better. HiSTrim demonstrates substantial KV compression alongside up to $2.82\times$ wall-clock inference speedups.